

グラフ理論用語集

1. 概要

本講座では、グラフに関する用語について整理します。

グラフ理論は数学分野の中でも、同じ用語が文献によって少し違った意味で使われているといったことが特に多く見られる分野です。本講座ではその中でも標準的と思われる定義を採用し、定義揺れが起こりやすい点については指摘したつもりですが、グラフ理論の文献を読む場合には改めて文献内の定義に注意して読むようにしてください。

なお AtCoder Algorithm Lectures では、特に説明されない場合、本用語集の定義を採用します。

たくさんの用語がありますが、一気にすべてを覚えるというような学習方法は想定していません。問題文や解説を読んでいて知らない用語が出てきたときに、その用語や関連する用語について確認するという使い方を想定しています。

競技プログラミングで登場する用語を高度なものまで網羅しているわけではありません。分からない用語が出てきたら、問題文や解説に定義が書かれていないかどうかとも再確認したり、インターネット検索なども活用してください。

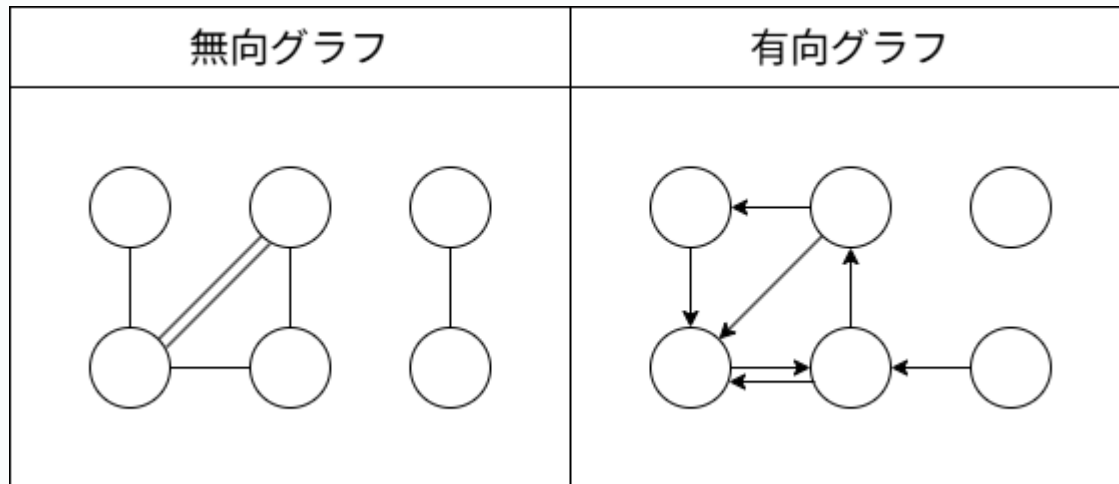
2. 前提知識

必要な前提知識は特にありません。

3. 基礎的な用語

3.1. 無向グラフ・有向グラフ

いくつかの**頂点** (vertex, node) と、いくつかの 2 頂点を結ぶ**辺** (edge) からなるものを**グラフ** (graph) といいます。辺の向きを区別しない場合には**無向グラフ** (undirected graph) といい、辺の向きを区別する場合には**有向グラフ** (directed graph, digraph) といいます。



より形式的には、グラフとは次の組として定義される場合があります（例えば [文献 1](#) を参照してください）。

- 頂点集合 V .
- 辺集合 E .
- E の各要素に対して、次を割り当てる規則。
 - 無向グラフの場合： V の要素からなる大きさ 2 の多重集合.
 - 有向グラフの場合： V の要素からなる順序対.

グラフ $G = (V, E)$ などと書いた場合、通常 V が頂点集合、 E が辺集合を指します。

無向グラフの辺を**無向辺** (undirected edge)、有向グラフの辺を**有向辺** (directed edge, arc) といいます。競技プログラミングの問題文では、これらを区別するために

- 「無向グラフが与えられる」「有向グラフが与えられる」
- 「辺 i は A_i, B_i を双方向に結んでいる」「辺 i は A_i, B_i をこの向きに一方方向に結んでいる」

などの表現がよく使われます。問題文を読む際にはこれらの表現を意識するとよいでしょう。

なお、本講座では「無向グラフ」「有向グラフ」のみを扱いますが、無向辺と有向辺が混在するグラフもしばしばグラフ理論の対象となります。

辺に割り当てられた 2 つの頂点を，その辺の**端点**（endpoint）といいます．有向辺の場合には，1 つめの頂点を**始点**（source），2 つめの頂点を**終点**（target）といいます．

3.2. 多重辺

同じ 2 頂点を同じように結ぶ辺を**多重辺**（multiple edges, parallel edges）といいます．より正確には，

- 無向グラフの場合： 2 つの辺に割り当てられた多重集合が一致する．
- 有向グラフの場合： 2 つの辺に割り当てられた頂点の順序対が一致する．

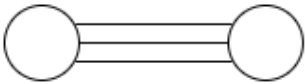
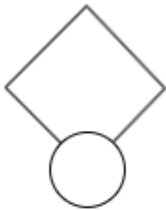
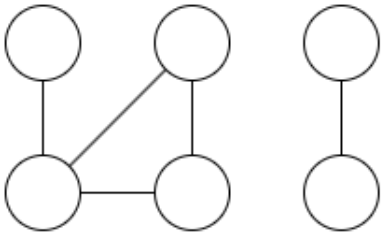
ときに，そのような辺を多重辺といいます．多重辺がないグラフでは，2 頂点を指定すれば辺が高々ひとつに決まるため，辺 uv ，辺 $u \rightarrow v$ のように辺を表すことがあります．

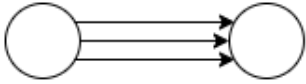
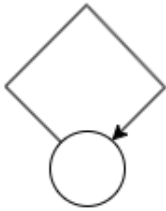
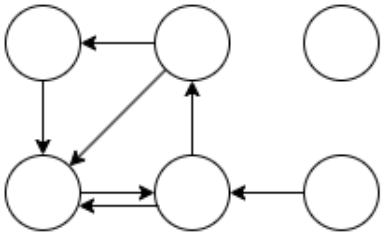
3.3. ループ

両端点が一致するような辺を，**ループ**または**自己ループ**（loop, self-loop）といいます．後述のサイクルとの混同に注意してください．

3.4. 単純グラフ

多重辺・ループが存在しないグラフを**単純グラフ**（simple graph）といいます．

無向グラフの多重辺	無向グラフのループ	単純無向グラフ
		

有向グラフの多重辺	有向グラフのループ	単純有向グラフ
		

単純無向グラフがグラフ理論における最も単純な設定です。文脈によっては単純無向グラフを単にグラフということもあります。

3.5. 隣接，接続

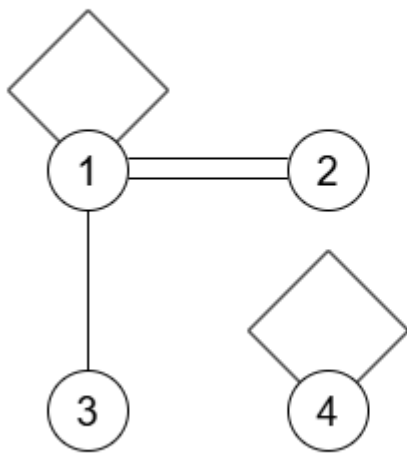
頂点 v が辺 e の端点のひとつであるとき， v は e に，あるいは e は v に**接続**（incident）しているといいます。

頂点 u, v がある辺 e の端点であるとき， u, v は**隣接**（adjacent）しているといいます。

3.6. 頂点の次数

v と接続している辺の個数を， v の**次数**（degree）といい， $\deg(v)$ と書きます。ただし，ループは v に 2 回接続していると考えて 2 回数えます。

グラフの次数



$$\deg(1) = 5$$

$$\deg(2) = 2$$

$$\deg(3) = 1$$

$$\deg(4) = 2$$

有向グラフの場合にはさらに、

- v を終点とする辺の個数を v の**入次数** (in-degree)
- v を始点とする辺の個数を v の**出次数** (out-degree)

といいます。

次数 0 の頂点を**孤立点** (isolated vertex) といいます。

有向グラフにおいて入次数 0 の頂点を**ソース頂点** (source vertex) といいます。有向グラフにおいて出次数 0 の頂点を**シンク頂点** (sink vertex) といいます。

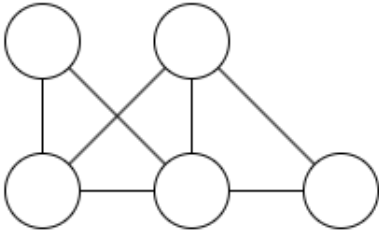
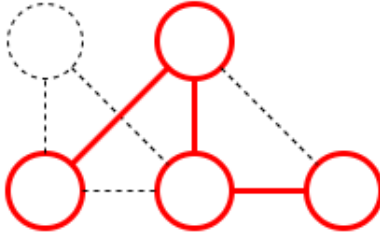
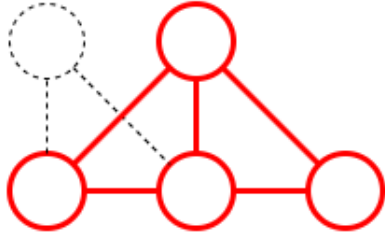
3.7. 部分グラフ・誘導部分グラフ

グラフの頂点、辺の一部（0 個や全体でもよい）からなるグラフを**部分グラフ** (subgraph) といいます。もとのグラフと異なる部分グラフを**真部分グラフ** (proper subgraph) といいます。

V をグラフ G の頂点集合とし、 S を V の部分集合とします。このとき、頂点集合を S とする最大の G の部分グラフ、つまり

- 頂点集合を S とする
- 両方の端点が S に含まれるような辺全体を辺集合とする

ような G の部分グラフを, G の S に関する**誘導部分グラフ** (induced subgraph) とい
い, $G[S]$ などと書きます.

G	G の部分グラフ	G の誘導部分グラフ
		

3.8. グラフの同型

グラフ $G_1 = (V_1, E_1)$ および $G_2 = (V_2, E_2)$ が**同型** (isomorphic) であるとは, 全単射 $f: V_1 \rightarrow V_2$ であって次の条件を満たすものが存在することをいいます.

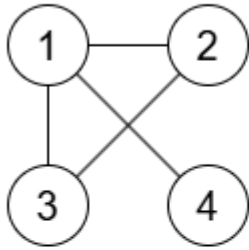
- 無向グラフの場合: 任意の $u, v \in V_1$ に対して, u と v を結ぶ G_1 の辺の個数が, $f(u)$ と $f(v)$ を結ぶ G_2 の辺の個数と等しい.
- 有向グラフの場合: 任意の $u, v \in V_1$ に対して, u から v に向かう G_1 の有向辺の個数が, $f(u)$ から $f(v)$ に向かう G_2 の有向辺の個数と等しい.

このような f を G_1 から G_2 への**同型写像** (isomorphism) といいます.

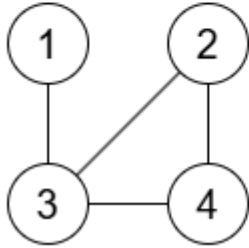
文献によっては, 頂点集合間の写像と辺集合間の写像の組によって同型写像を定義する場合もあります.

グラフの同型写像の例

G_1



G_2



$$f: \{1, 2, 3, 4\} \longrightarrow \{1, 2, 3, 4\}$$

$$f(1) = 3$$

$$f(2) = 2$$

$$f(3) = 4$$

$$f(4) = 1$$

4. ウォーク

4.1. ウォーク

ある頂点から出発して、辺を通して何回か移動することを、グラフ上の**ウォーク** (walk) や**歩道**といいます。より形式的にはウォークとは、

- 非負整数 n ,
- 頂点の列 $(v_0, v_1, \dots, v_n)$,
- 辺の列 $(e_0, e_1, \dots, e_{n-1})$

の組であって、以下の条件を満たすものをいいます。

- 無向グラフの場合： e_i は v_i, v_{i+1} を結ぶ無向辺.
- 有向グラフの場合： e_i は v_i から v_{i+1} への有向辺.

v_0 をこのウォークの**始点** (start vertex), v_n をこのウォークの**終点** (end vertex) といいます。またこのウォークを v_0 から v_n へのウォークといいます。

v_i, e_i がウォークに含まれることを、ウォークが v_i, e_i を**通る** (traverse) といいます。

辺の個数 n をこのウォークの**長さ** (length) といいます. 長さ 0 のウォーク (1 頂点のみからなるウォーク) も考えています. 長さ無限大のウォークを考えることもありますが, その場合の定義は同様なので省略します.

4.2. パス

ウォークの定義において $v_0, v_1, \dots, v_n$ がすべて異なるとしたものを**パス** (path) や**道**といいます. ウォークとの違いを明示するために, **単純パス** (simple path) という用語が使われることもあります.

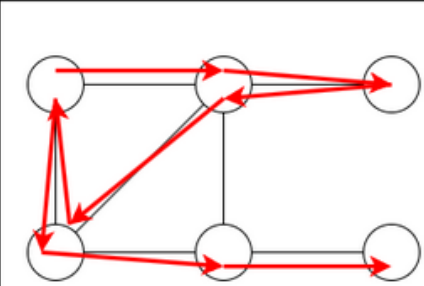
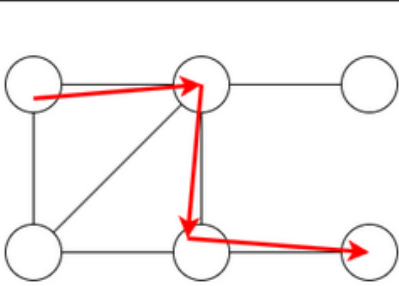
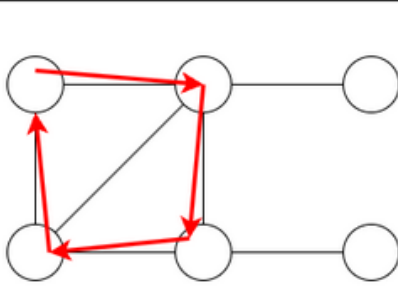
文献や問題文によっては, ウォークのことをパスと呼んでいることもあります (流儀が異なる場合や, 単に使い分けが意識されていない場合もあります).

なお, 後述の**パスグラフ**を指してパスと呼ぶことも多くあります.

4.3. サイクル

ウォークの定義において, $n > 0$ であり, $v_0, v_1, \dots, v_{n-1}$ および $e_0, e_1, \dots, e_{n-1}$ がすべて異なり, $v_0 = v_n$ が成り立つとしたものを**サイクル** (cycle) や**閉路**といいます.

なお, 後述の**サイクルグラフ**を指してサイクルと呼ぶことも多くあります.

ウォーク	パス	サイクル
		

4.4. 小道, 回路

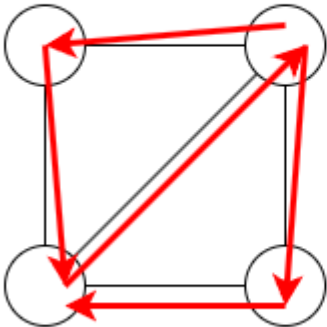
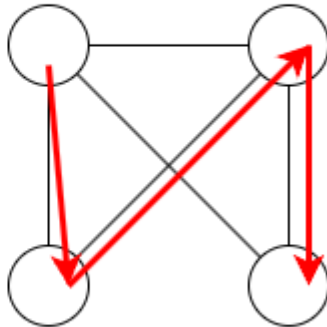
ウォークであって同じ辺を高々 1 度までしか通らないものを, **小道** (trail) といいます. そのうち始点と終点が一致するものを**回路** (circuit) といいます.

4.5. オイラーウォーク

グラフのすべての辺をちょうど 1 回ずつ通るウォークを、**オイラーウォーク** (Eulerian walk) や**オイラー小道** (Eulerian trail) といいます。そのうち始点と終点が一致するものを、**オイラーサーキット** (Eulerian circuit) や**オイラー回路**といいます。

4.6. ハミルトンパス

グラフのすべての頂点を通るパスを**ハミルトンパス** (Hamiltonian path) といいます。グラフのすべての頂点を通るサイクルを**ハミルトンサイクル** (Hamiltonian cycle) といいます。

オイラーウォーク	ハミルトンパス
	

注意

「何らかの条件を満たすウォーク」に該当する概念はたくさんあり、使い分けが複雑で覚えにくいです。AtCoder Algorithm Lectures では「**ウォーク**」「**パス**」「**サイクル**」の 3 つの用語を積極的に用います。それ以外の用語を用いる場合には、その講座内で定義を再掲する予定です。

ウォークやパスに関する用語や、その日本語訳は文献による定義揺れもしばしば見られます。用語の使い方について詳しく確認したい場合には、[文献 1](#)、[文献 4](#)、[文献 6](#)などを参照してください。

5. 連結性

5.1. 連結成分，強連結成分

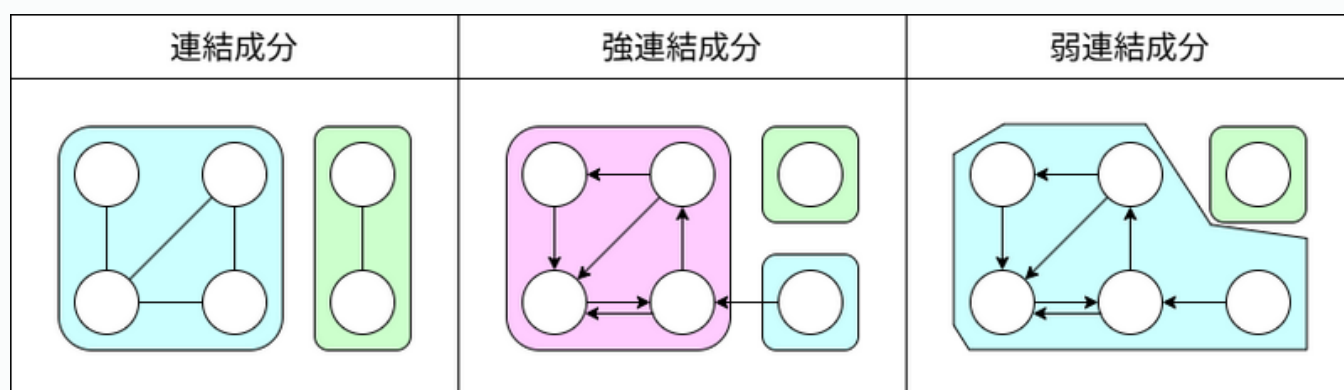
ある頂点 v と，ウォークによって互いに到達可能な頂点全体を，

- 無向グラフの場合： v の**連結成分** (connected component)
- 有向グラフの場合： v の**強連結成分** (strongly connected component)

といいます。2つの連結成分は共通の頂点を持つならば完全に一致することが証明でき，グラフの頂点集合は連結成分に分割されます。強連結成分についても同様です。

ここでは連結成分（強連結成分）を頂点集合として定義しましたが，その頂点集合による誘導部分グラフのことを連結成分（強連結成分）と呼ぶこともあります。

有向グラフにおいて，有向辺をすべて無向辺と見なして得られるグラフでの連結成分を考えることもあります。この場合の連結成分は，強連結成分との区別を強調して**弱連結成分** (weakly connected component) ということがあります。



5.2. 連結グラフ，強連結グラフ

連結成分がちょうどひとつであるような無向グラフを**連結グラフ** (connected graph) といいます。

強連結成分がちょうどひとつであるような有向グラフを**強連結グラフ** (strongly connected graph) といいます。

注意

本講座では上の定義のように，頂点数が0のグラフ（**空グラフ**）について，**連結ではないとする定義を採用しました**。グラフ理論に限らず数学の他分野でも，連結性の概念は空の場合の

扱いについて揺れがあり，定義は文献によって異なるので，特に注意して確認してください．

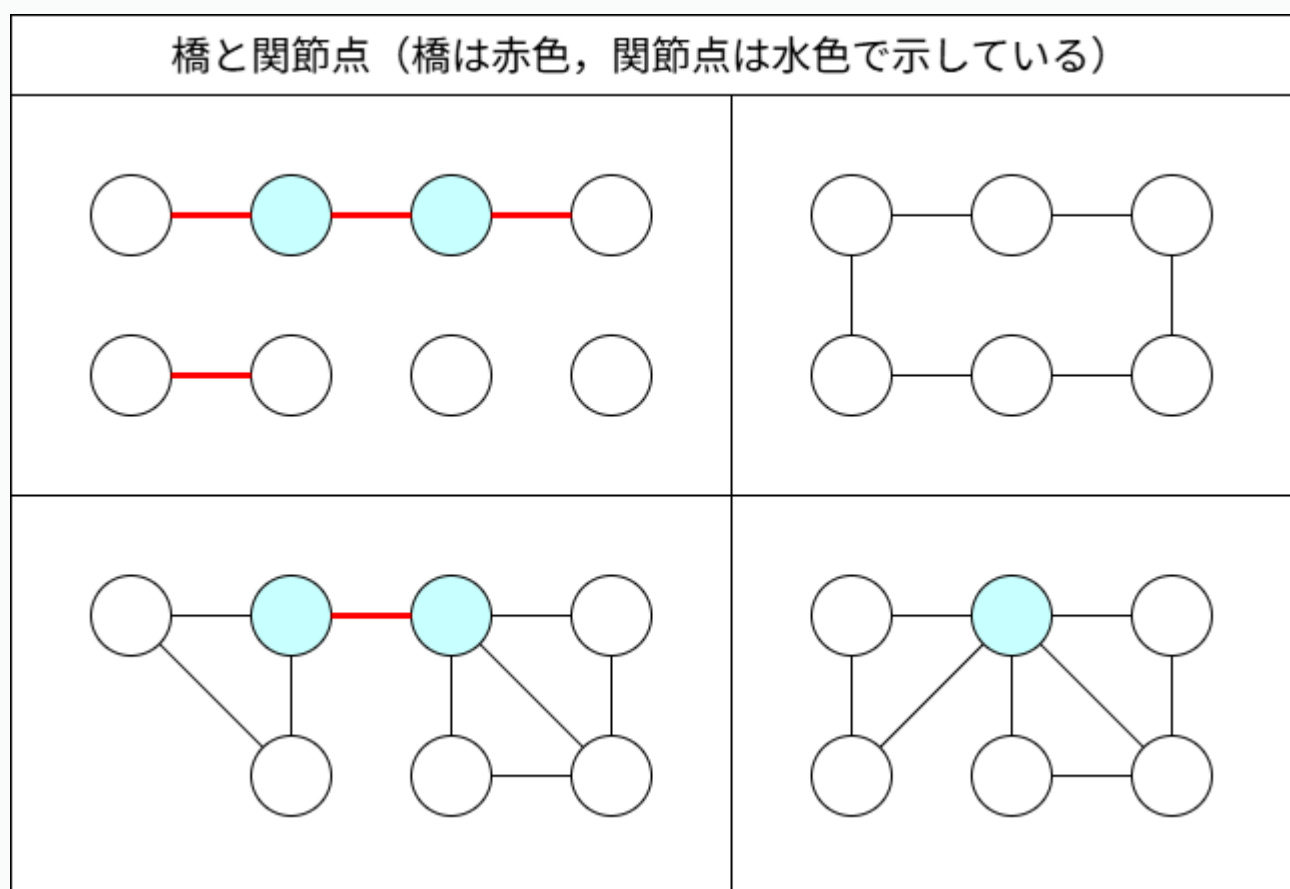
5.3. 橋

無向グラフ G の辺 e について， e を取り除くことにより連結成分数が増えるとき， e を G の橋 (bridge) といいます．

辺 e が橋であることと， e を含むサイクルが存在しないことは同値です．

5.4. 関節点

無向グラフ G の頂点 v について， v および v に接続するすべての辺を取り除くことにより連結成分数が増えるとき， v を G の関節点 (articulation point, cut vertex) といいます．



6. 重要な無向グラフのクラス

6.1. 空グラフ

頂点集合，辺集合が空集合であるグラフを**空グラフ**（empty graph, null graph）といいます。

文献や文脈によっては，辺のないグラフ（edgeless graph），つまり辺集合が空集合であるグラフを空グラフと呼ぶこともあるようです。

6.2. 完全グラフ

すべての異なる 2 頂点間にちょうどひとつの辺があるような無向グラフを**完全グラフ**（complete graph）といいます。

なお，英語で perfect graph と呼ばれるグラフのクラスもあるのですが，これは日本語では理想グラフと訳されており，完全グラフとは全く別の概念なので注意してください。本講座では理想グラフの定義は扱いません。

6.3. パスグラフ

すべての頂点・辺を通るパスが存在するようなグラフを**パスグラフ**（path graph）あるいは単に**パス**といいます。

パスグラフと，ウォークとしてのパスは，区別の重要性が低い場合には曖昧に使われることも多いです。AtCoder Algorithm Lectures では，区別の重要性が高い場合には，「ウォークとして」「グラフとして」といった表現を心がける予定です。

6.4. サイクルグラフ

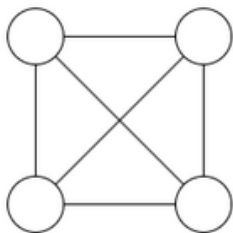
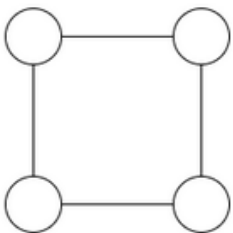
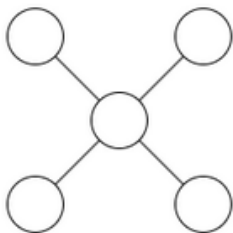
すべての頂点・辺を通るサイクルが存在するようなグラフを**サイクルグラフ**（cycle graph）あるいは単に**サイクル**といいます。

サイクルグラフと，ウォークとしてのサイクルは，区別の重要性が低い場合には曖昧に使われることも多いです。AtCoder Algorithm Lectures では，区別の重要性が高い場合には，「ウォークとして」「グラフとして」といった表現を心がける予定です。

6.5. スターグラフ

n を非負整数とすると、 $n + 1$ 頂点からなるグラフであって、ひとつの頂点とその他の n 頂点を結ぶ n 辺からなるグラフを**スターグラフ** (star graph) あるいは単にスターといいます。

日本の競技プログラミングでは**ユニグラフ**という俗称もあります (参考: [AGC009D](#))。

完全グラフ	パスグラフ	サイクルグラフ	スターグラフ
			

注意

n 頂点の完全グラフ、パスグラフ、サイクルグラフ、スターグラフについて、 K_n , P_n , C_n , S_n という記号を使う場合があります。

ただし、パスグラフ、スターグラフについては頂点数と辺数のどちらを添字 n とするかについて、文献による定義揺れがあるので注意してください。

AtCoder Algorithm Lectures では、本講座を含めて、これらの記号を講座内での定義なしに使用することはしません。

6.6. 木

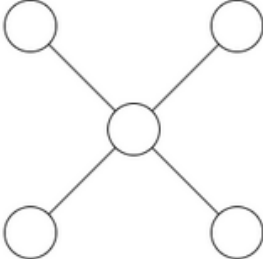
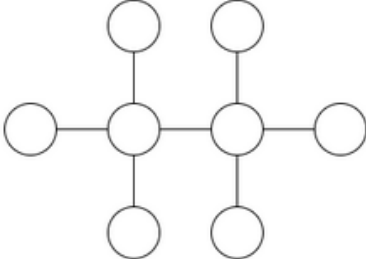
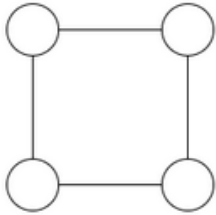
n を正整数とすると、 n 頂点 $n - 1$ 辺からなる連結グラフを**木** (tree) といいます。他にも同値な定義はいくつか知られており、例えば

- 空ではないグラフであって、任意の 2 頂点 u, v に対して、 u から v へのパスが唯一存在するもの。
- n 頂点 $n - 1$ 辺からなるグラフであってサイクルを含まないもの。

などの定義が可能です。

空グラフは木ではありません。例えば単に「任意の 2 頂点間にパスが唯一存在する」という条件を木の定義とすると誤りになるので注意してください。

パスグラフやスターグラフは木の代表例です。

木	木	木	木ではない
			

木の 2 頂点 u, v に対して、 u から v へのパスが唯一存在します。このパスを、パス uv やパス $u \rightarrow v$ などと書きます。

6.7. 森

すべての連結成分が木であるようなグラフを**森**（forest）といいます。

無向グラフが森であることと、サイクルを含まない（acyclic）ことは同値です。

空グラフは森です。木は森です。

6.8. 二部グラフ

頂点集合を V_1, V_2 に分割して、 V_1 の頂点同士・ V_2 の頂点同士を結ぶ辺が存在しないようにできるようなグラフを**二部グラフ**（bipartite graph）といいます。

同じ意味ですが、次のような表現をすることもよくあります：

- 頂点を色 1、色 2 に塗り分けて、同色の頂点を結ぶ辺が存在しないようにできる。

二部グラフを V_1, V_2 に分割する方法は一般には複数存在することに注意してください。例えば辺のないグラフにおいては頂点集合の任意の分割が条件を満たします。

頂点集合の分割方法を含めたデータを考える際に、「二部グラフ $G = (V_1, V_2, E)$ 」 というような表現が使われることもあります。

森は二部グラフです。サイクルは長さが偶数であるとき、そのときに限り二部グラフです。

あるグラフが二部グラフであることは、長さが奇数のサイクルを含まないことと同値であることが知られています。

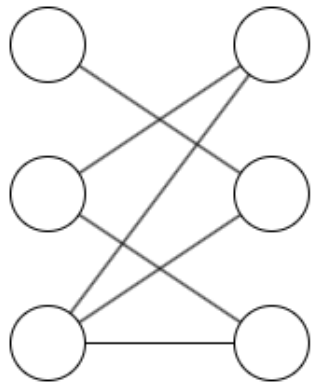
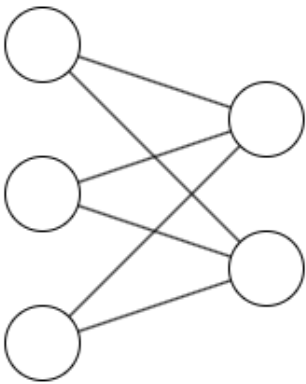
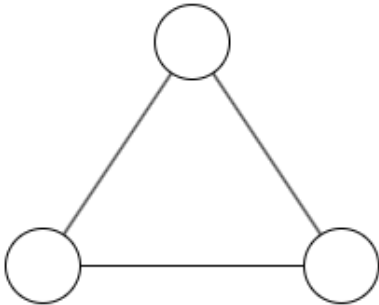
6.9. 完全二部グラフ

頂点集合を V_1, V_2 に分割して、

- V_1 の頂点同士・ V_2 の頂点同士を結ぶ辺が存在しない
- 任意の V_1 の頂点と V_2 の頂点について、それらを結ぶ辺がちょうどひとつ存在する

ようにできるグラフを**完全二部グラフ** (complete bipartite graph) といいます。 $|V_1| = n, |V_2| = m$ にとれるような完全二部グラフを $K_{n,m}$ と書くことがあります。

完全二部グラフは二部グラフです。スターグラフは完全二部グラフです。

二部グラフ	完全二部グラフ $K_{3,2}$	二部グラフではない
		

6.10. 多部グラフ

頂点集合を $V_1, V_2, \dots, V_k$ に分割して、任意の i に対して V_i の頂点同士を結ぶ辺が存在しないようにできるようなグラフを **k 部グラフ** (k -partite graph) といいます。 $k = 2$ の場

合が二部グラフです.

二部グラフの場合と同様に, **完全 k 部グラフ** (complete k -partite graph) も定義されます.

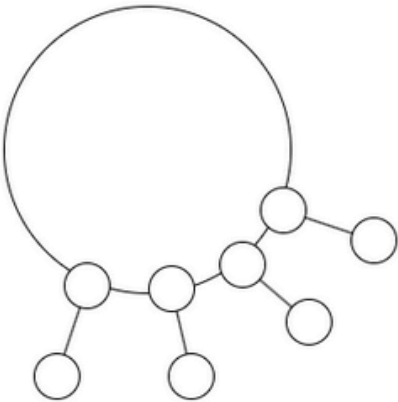
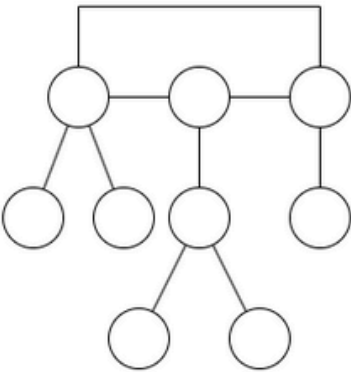
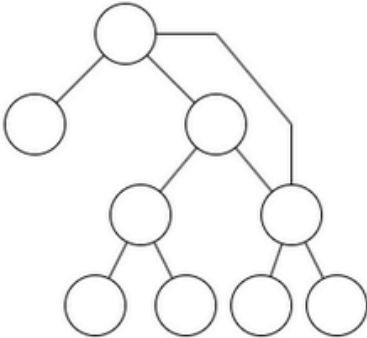
6.11. なもりグラフ (Unicyclic グラフ)

n を正整数とすると, n 頂点 n 辺の連結グラフを**Unicyclic グラフ** (unicyclic graph) や**なもりグラフ**といいます.

木にひとつ辺を追加することにより得られるグラフで, 部分グラフとしてちょうどひとつのサイクルを含むという特徴があります.

なもりグラフという呼び方は日本競技プログラミング発祥であり, AtCoder の問題名でも何度も採用されています (例えば [AGC004F](#)). この呼称は漫画家なもり氏の自画像に由来すると言われています.

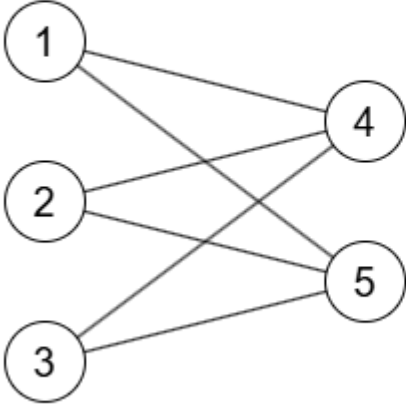
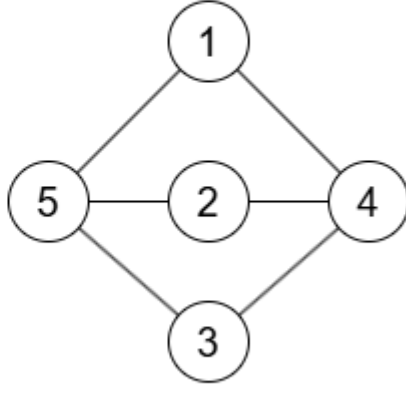
Unicyclic グラフの日本語訳としては他に, 「単一閉路グラフ」「単一サイクルグラフ」「ユニサイクルグラフ」などが確認できましたが, いずれも少数の使用例しか見つかりませんでした.

なもりグラフ	なもりグラフ	なもりグラフ
		

6.12. 平面グラフ, 平面的グラフ

平面上の点を頂点, 頂点同士を結ぶ線分を辺とするグラフであって, 辺同士は端点以外では交わらない線分であるものを**平面グラフ** (plane graph) といいます. 辺については, 折れ線や曲線を用いて定義されることもあります.

平面グラフと同型であるグラフを**平面的グラフ**（planar graph）といいます。平面的グラフに対して、平面グラフへの同型をとることを、**描画**（drawing）や**埋め込み**（embedding）といいます。

平面的グラフ	その描画である平面グラフ
	

7. 無向グラフに関する諸概念

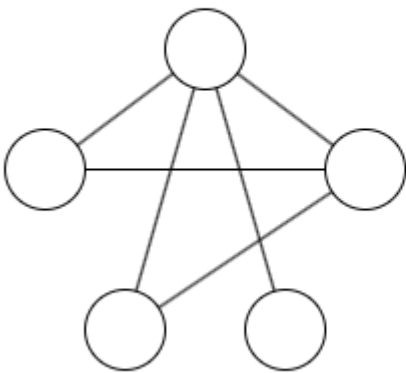
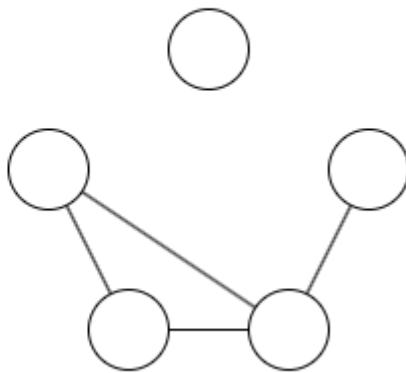
7 節において、単にグラフと書けば無向グラフを表すこととします。

7.1. 補グラフ

頂点集合が等しい単純グラフ G_1, G_2 について、次が成り立つとき、 G_2 は G_1 の**補グラフ**（complement graph）といいます。

- 任意の相異なる 2 頂点 u, v に対して、 G_1 に辺 uv が存在**する**ことと G_2 に辺 uv が存在**しない**ことは同値である。

G_2 が G_1 の補グラフであるとき、 G_1 は G_2 の補グラフです。

G_1	G_1 の補グラフ G_2
	

7.2. 独立集合

グラフにおいて、頂点からなる集合であって、どの頂点もループに接続しておらず、どの相異なる 2 頂点も隣接していないものを**独立集合** (independent set) といいます。

空集合は独立集合です。 $S_1 \subseteq S_2$ で S_2 が独立集合ならば S_1 も独立集合です。

要素数が最大の独立集合を**最大独立集合** (maximum independent set) といいます。

7.3. クリーク

単純グラフにおいて、頂点からなる集合で、どの 2 頂点も隣接しているものを**クリーク** (clique) といいます。頂点からなる集合がクリークであることは、その補グラフにおいて独立集合であることと同値です。

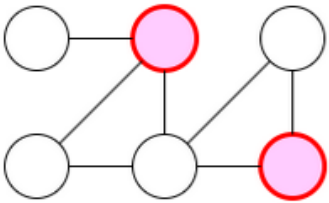
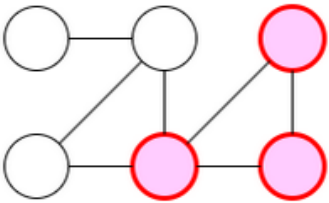
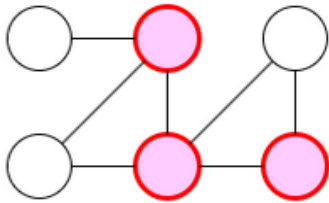
要素数が最大のクリークを**最大クリーク** (maximum clique) といいます。

7.4. 頂点被覆

グラフにおいて、頂点からなる集合であって、任意の辺がその集合内の頂点に接続しているものを**頂点被覆** (vertex cover) といいます。

単純グラフにおいて、頂点からなる集合が頂点被覆であることは、その補集合が独立集合であることと同値です。

要素数が最小の頂点被覆を**最小頂点被覆**（minimum vertex cover）といいます。

独立集合	クリーク	頂点被覆
		

7.5. 辺被覆

グラフにおいて、辺からなる集合であって、任意の頂点とその集合内の辺に接続しているものを**辺被覆**（edge cover）といいます。

孤立点を持たないグラフにおいて辺被覆は存在します（例えばすべての辺からなる集合が条件を満たします）。要素数が最小の辺被覆を**最小辺被覆**（minimum edge cover）といいます。

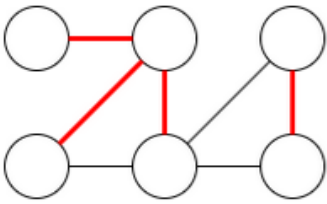
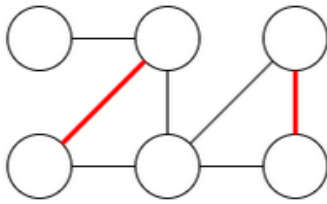
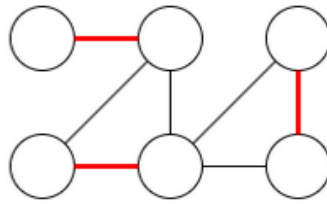
7.6. マッチング

グラフにおいて、辺からなる集合であって、ループを含まず、どの 2 つの辺も端点を共有しないものを**マッチング**（matching）といいます。

要素数が最大のマッチングを**最大マッチング**（maximum matching）といいます。

マッチングでも辺被覆でもある辺集合を**完全マッチング**（perfect matching）といいます。

完全マッチングは最大マッチングでもあります。最大マッチングは必ず存在しますが、完全マッチングは存在するとは限りません。例えば頂点数が奇数のグラフには完全マッチングが存在しません。

辺被覆	マッチング	完全マッチング
		

7.7. 頂点彩色

ループのないグラフにおいて、隣接する 2 頂点が同じ色を持たないように、頂点に色を割り当てることを**頂点彩色**（vertex coloring）といいます。頂点彩色に必要な色数の最小値をそのグラフの**彩色数**（chromatic number）といいます。

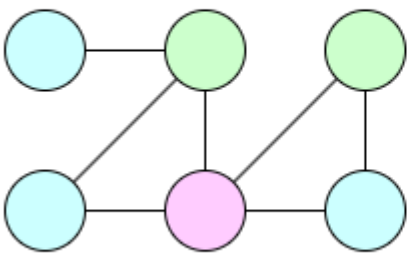
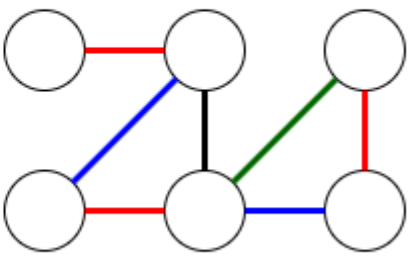
ループを持つグラフについては、彩色数は定義しない場合や、 ∞ と定義する場合があります。

グラフが二部グラフであることと、彩色数が 2 以下であることは同値です。

7.8. 辺彩色

ループのないグラフにおいて、同じ頂点に接続する 2 辺が同じ色を持たないように、辺に色を割り当てることを**辺彩色**（edge coloring）といいます。辺彩色に必要な色数の最小値をそのグラフの**辺彩色数**（edge-chromatic number, chromatic index）といいます。

ループを持つグラフについては、辺彩色数は定義しない場合や、 ∞ と定義する場合があります。

点彩色	辺彩色
	

8. 木の用語

8.1. 根，根付き木

木において特別にひとつ選んだ頂点を**根**（root）といい，根が選ばれている木を**根付き木**（rooted tree）といいます．より形式的には，根付き木とは木と根として選ばれた頂点の組です．

木に対するアルゴリズムのほとんどは，（問題文で根として選ぶべき頂点が指定されていない場合であっても）木を根付き木として扱うことでアルゴリズムを構築します．

以下で述べる用語も，一部の例外を除き根付き木に対して定義されています．

8.2. 親，子

根付き木において， v を根以外の頂点とすると， v と根を結ぶパスが唯一存在します．このパスにおいて v の次に通る頂点を v の**親**（parent）といいます．

w が v の親であるとき， v は w の**子**（child）といいます．

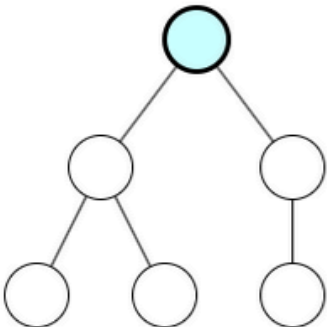
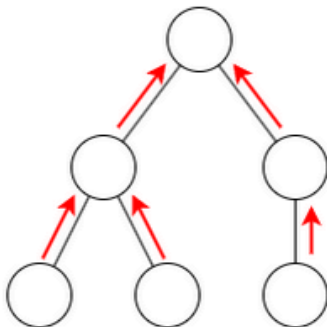
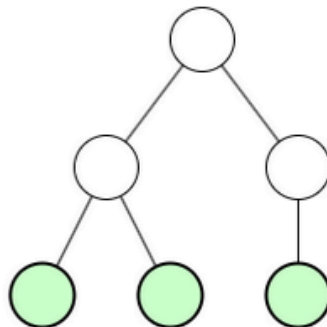
AtCoder Algorithm Lectures では，根付き木を図示するときには，基本的に根を最も上に描き，それ以外の頂点についても親が上側，子が下側になるように図示します．

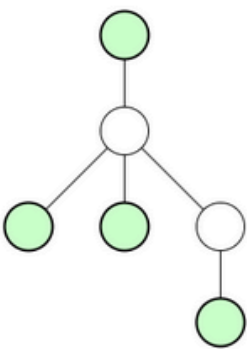
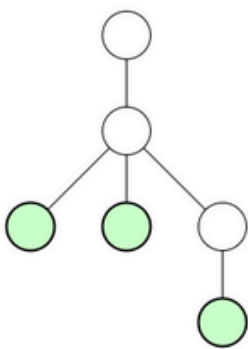
8.3. 葉

次の 2 通りの定義があります。

- 木において、次数 1 の頂点を**葉** (leaf) という。
- 根付き木において、子を持たない頂点を**葉** (leaf) という。

これらの定義は、根付き木において根の次数が 1 である場合や、頂点数が 1 である場合に差異が生じます。競技プログラミングで根付き木の葉を扱う問題文では、通常これらを区別するための定義が書かれています。

根	子から親	葉
		

木の葉	根付き木の葉	木の葉	根付き木の葉
			

8.4. 深さ

根付き木の頂点 v に対して、 v と根を結ぶパスの長さを v の**深さ** (depth) といいます。

特に AtCoder Algorithm Lectures では、根の深さを 0 とする定義を採用しています。根の深さを 1 と定義する問題文などもあるので注意してください。

8.5. 祖先

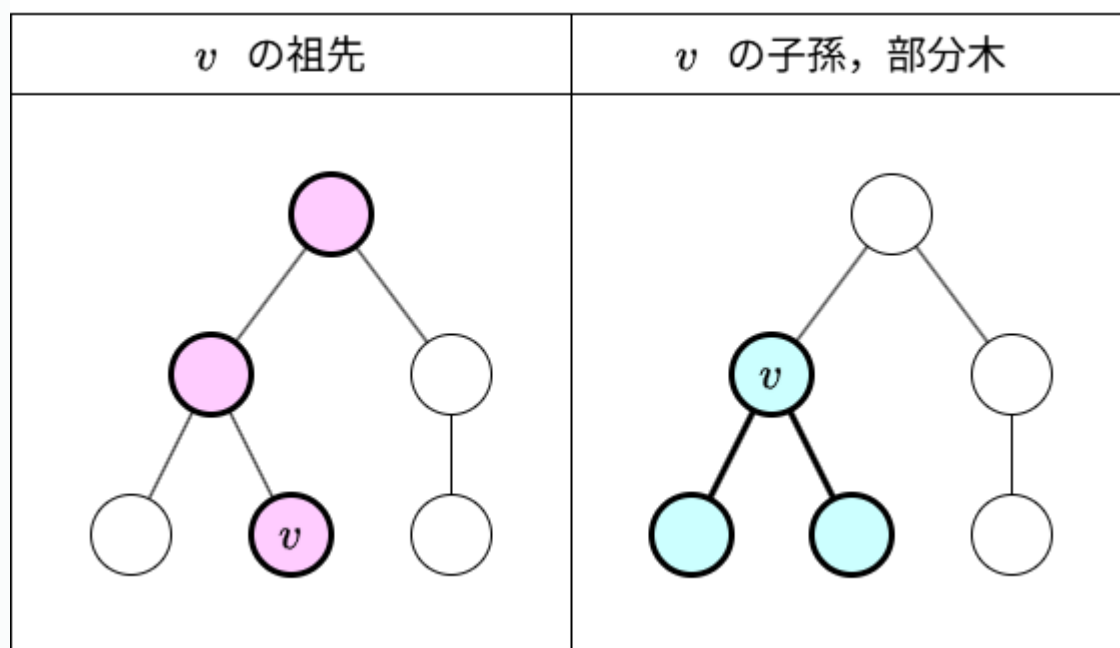
根付き木の頂点 v に対して、 v と根を結ぶパスが通る頂点を v の**祖先** (ancestor) といいます。定義の通り、通常は v 自身も v の祖先であるとして扱います。

8.6. 子孫，部分木

u が v の祖先であるとき、 v は u の**子孫** (descendant) であるといいます。

根付き木において、頂点 v の子孫全体、あるいは子孫全体による誘導部分グラフを v の**部分木** (subtree) といいます。「部分木 v 」と書くこともあります。

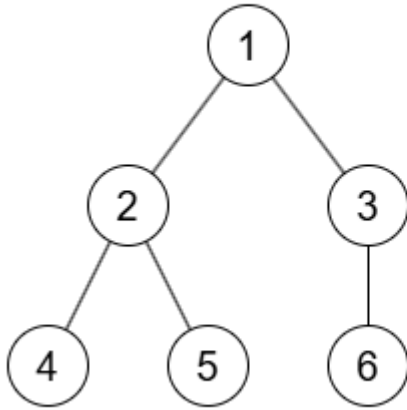
木の部分グラフであって木であるものを部分木と呼ぶ用例もあるので注意してください。



8.7. LCA（最小共通祖先）

根付き木の頂点 u, v に対して、これらに共通の祖先のうちで深さが最大であるものを**最小共通祖先** (lowest common ancestor, LCA) といいます。以降、最小共通祖先を **LCA** と略記し、 u, v の最小共通祖先を $\text{LCA}(u, v)$ と書きます。

根付き木の LCA（最小共通祖先）



$$\text{LCA}(4, 1) = 1$$

$$\text{LCA}(4, 2) = 2$$

$$\text{LCA}(4, 3) = 1$$

$$\text{LCA}(4, 4) = 4$$

$$\text{LCA}(4, 5) = 2$$

$$\text{LCA}(4, 6) = 1$$

8.8. 高さ

根付き木において、頂点の深さの最大値を**高さ**（height）といいます。

v の部分木の高さを v の**高さ**といいます。根付き木の高さは根の高さと一致します。

8.9. 根付き森

すべての連結成分が根付き木であるような森を、**根付き森**（rooted forest）といいます。根付き森についても、親、子、深さ等の上で定義した用語のいくつかを同様に用います。

8.10. 二分木

根付き木であって、任意の頂点が 2 つ以下の子を持つものを考えます。このとき、各頂点の子に、左の子・右の子のいずれかを重複なく割り当てることができます。このように、各頂点の子に左・右を割り当てた根付き木を**二分木**（binary tree）といいます。

なお文献によっては、任意の頂点が 2 つ以下の子を持つような根付き木を二分木と定義することもあります。

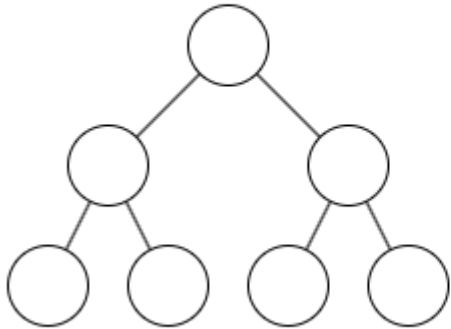
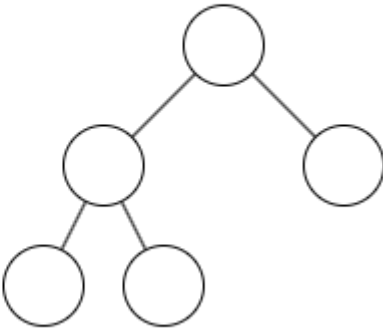
すべての頂点の子の個数が 0 または 2 であるような二分木を**全二分木**（full binary tree）といいます。

8.11. 完全二分木

高さ h の全二分木であって、すべての葉の深さが h であるものを**完全二分木**（perfect binary tree）といいます。この木の頂点数は $2^{h+1} - 1$ です。

高さ h の二分木であって、深さ $0, 1, \dots, h - 1$ では頂点が最大数存在し、深さ h の頂点が左から詰まっているものも**完全二分木**（complete binary tree）といいます。この木の頂点数は 2^h 以上 $2^{h+1} - 1$ 以下です。

日本語では 2 通りの意味で使われていることに注意してください（参考：[ABC264Ex](#), [ABC321E](#)）。

完全二分木（perfect binary tree）	完全二分木（complete binary tree）
	

9. 有向グラフの用語

9.1. DAG（有向非巡回グラフ）

有向グラフであって、サイクルを含まないものを**有向非巡回グラフ**（directed acyclic graph, DAG）といいます。以降、有向非巡回グラフを **DAG** と略記します。なお、すべての強連結成分の頂点数が 1 であるということと同値です。

例えば頂点に整数のラベルがつけられていて、任意の辺 $u \rightarrow v$ に対して $u < v$ が成り立つとき、このグラフは DAG になります。競技プログラミングの問題文では、この形の制約によってグラフが DAG であることを表している場合があります。

9.2. トポロジカルソート

$G = (V, E)$ を有向グラフとすると、 V の頂点を $v_1, v_2, \dots$ と並べる方法であって、有向辺 $v_i \rightarrow v_j$ が存在するならば $i < j$ が成り立つものを、 G の**トポロジカルソート** (topological sort) といいます。

有向グラフに対して、トポロジカルソートが存在することと、DAG であることは同値であることが証明できます。

9.3. 有向木

根付き木の辺に、根から遠ざかる方向に向きをつけたものを**有向木** (directed tree, arborescence) といいます。

根に向かう方向に向きをつけたものも**有向木**といい、必要な場合には辺が根から出る、根に入るという方向に応じて **out-tree**, **in-tree** などの用語で区別されることがあります。

木の各辺にそれぞれ向きを付けたものを有向木と呼ぶ文献もあります。

DAG	有向木 (out-tree)	有向木 (in-tree)

9.4. Functional グラフ

すべての頂点の出次数が 1 であるようなグラフを **Functional グラフ** (functional graph) といいます。

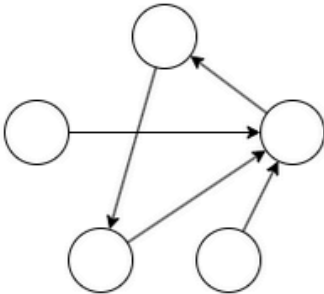
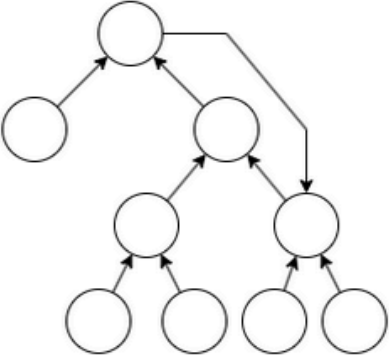
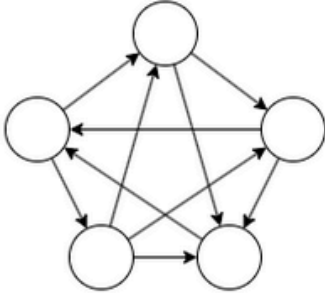
辺の向きを無視すると、各連結成分で頂点数と辺数が等しくなります。したがって、各連結成分はなもりグラフになります。

日本語では「関数グラフ」などの用例もあるようですが、解析学における関数のグラフと区別がしづらく紛らわしいこともあり、普及していません。英語表記のまま Functional グラフという用例が日本でも最も普及しているようなので、AtCoder Algorithm Lectures でもそのように表記します。

9.5. トーナメント

単純有向グラフであって、任意の相異なる 2 頂点 u, v に対して $u \rightarrow v$ または $v \rightarrow u$ のうちちょうど一方の辺が存在するものを**トーナメント** (tournament) といいます。

日本語での日常用語からの推測だと、全二分木のようなものを想像してしまう方も多いと思うので、注意してください。また文献や問題によっては、「完全有向グラフ」と呼ばれていることもあります。この用語は多くの場合、任意の相異なる 2 頂点 u, v に対して $u \rightarrow v, v \rightarrow u$ の**両方**が存在する有向グラフを指します。

Functional グラフ	Functional グラフ	トーナメント
		

10. 関連問題

特にありません。

11. まとめ

本講座では、グラフに関する用語について整理しました。繰り返し指摘した通り、グラフ理論は、同じ用語が文献によって少し違った意味で使われているといったことが特に多く見られる分野なので、注意してください。

本講座で整理した概念のほとんどは、今後の AtCoder Algorithm Lectures でも扱う機会があると思うので、楽しみにしててください。

12. 参考文献

1. Wikipedia. グラフ理論. <https://ja.wikipedia.org/wiki/グラフ理論>. (閲覧日: 2026-03-31).
2. Reinhard Diestel. Graph Theory. Springer. 2017. 5th ed.. Graduate Texts in Mathematics. Vol. 173. ISBN: 978-3-662-53621-6.
<https://link.springer.com/book/10.1007/978-3-662-53622-3>.
3. Alexander Schrijver. Combinatorial Optimization. Springer. 2003. ISBN: 978-3-540-44389-6. <https://link.springer.com/book/9783540443896>.
4. Wikipedia. 道 (グラフ理論). [https://ja.wikipedia.org/wiki/道_\(グラフ理論\)](https://ja.wikipedia.org/wiki/道_(グラフ理論)). (閲覧日: 2026-03-31).
5. Wikipedia. 二分木. <https://ja.wikipedia.org/wiki/二分木>. (閲覧日: 2026-03-31).
6. 37zigen. 歩道、道、小道、閉路、回路の定義. URL : <https://37zigen.com/graph-definition/>. (2026-06-11 現在リンク切れ).

トップページ : [AtCoder Algorithm Lectures](#)

質問・誤植報告・補足情報など : [Discord サーバー](#)

AtCoderは、世界最高峰の競技プログラミングサイトです。 リアルタイムのオンラインコンテストで競い合うことや、 5,000以上の過去問にいつでもチャレンジすることができます。



[ライセンス表記](#)

AtCoderについて

[AtCoderとは？](#)

[競技プログラミングとは？](#)

[レーティングのしくみ](#)

[アクセスガイド](#)

[お知らせ](#)

資料請求

[AtCoder](#)

[AtCoderJobs](#)

各サービス

[AtCoder](#)

[AtCoderJobs](#)

[アルゴリズム実技検定](#)

[AtCoderCareerDesign](#)

AtCoder株式会社について

[会社概要](#)

[FAQ](#)

[プライバシーポリシー](#)

[利用規約](#)